Journal Name

Crossmark

RECEIVED
dd Month yyyy

REVISED
dd Month yyyy

ARTICLE TYPE

Reconstructing Backpropagation from Forward Fluctuations in Noise-modulated Neural Networks

Shuhei Ikemoto[1,*]

[1]Graduate School of Life Science and Systems Engineering, Kyushu Institute of Technology, Kitakyushu, Japan
[*]Author to whom any correspondence should be addressed.

E-mail: s.ikemoto@ieee.org

Keywords: stochastic resonance, weight transport, weight mirror, covariance credit assignment

Abstract

A Noise-modulated Neural Network (NNN) is a neural network that can only learn and infer by adding noise, utilizing the noise as a computational resource rather than eliminating it as a disturbance. Thanks to the injected noise, this model can learn efficiently using backpropagation while employing spike-like signals. However, backpropagation requires a reverse path via the transposed weight matrices, known as the weight transport problem. Consequently, relying on backpropagation for learning undermines its biological or neuromorphic plausibility. Many attempts have been made to avoid this weight transport and learn using only the forward pass. However, most of these approaches replace the gradients of backpropagation with alternative objective functions or fixed random weights, which inevitably lead to learning instability and reduced accuracy. In this paper, we demonstrate that the structure of backpropagation in the NNN can be reconstructed solely from forward-pass statistics. By using the weight mirror that estimates the weight matrix from the covariance between a previous-layer unit's output and the next-layer unit's input, and combining this with local differential estimation within the units, the output error can be propagated recursively along the computational graph. We reconstruct the weight updates of backpropagation without using any transposed weight readouts. The gradient of the proposed learning rule is an empirically near-unbiased estimator that matches the true gradient, and by combining it with Adam updates for each local weight, we confirmed that it achieves final accuracy on par with backpropagation for simple regression tasks. Furthermore, when the injected noise follows a uniform distribution, the local operations degenerate to the polynomial and comparator levels, the entire system, including the learning rule, becomes well-suited for digital circuit implementation. These results demonstrate that in the NNN, noise serves as a resource not only for inference but also for reconstructing backpropagation.

1 Introduction

In biological nervous systems, neural noise, such as trial-to-trial variability in spike timing and firing rates, has long been considered to play functional roles rather than merely degrading signals. Representative examples include stochastic resonance, where noise helps detect weak signals by crossing a threshold [1, 2], trial-to-trial variability promoting exploration and learning [3, 4], and stochastic firing serving as the foundation for probabilistic inference and sampling in the brain [5, 6]. These examples share a perspective that treats noise as a computational resource rather than as a nuisance [7, 8]. Based on this view, establishing a mathematical model that essentially and fully utilizes noise as a computational resource is a key target for neuromorphic computing.

The Noise-modulated Neural Network (NNN) [9, 10, 11] integrates this perspective into artificial neural networks. Its basic elements are stochastic binary units called crossing activation functions, which fire only when the input crosses a threshold. The model generates no spikes under constant, noise-free inputs. Information transmission via spike-like binary signals occurs only when noise is injected into the units. The continuous information of the input vector is probabilistically encoded into these firing events. Functions equivalent to those of a conventional neural network (NN) are achieved by decoding the probabilistic features of the output vector through ensemble averaging. Furthermore, both the expected output and the expected derivative of the crossing

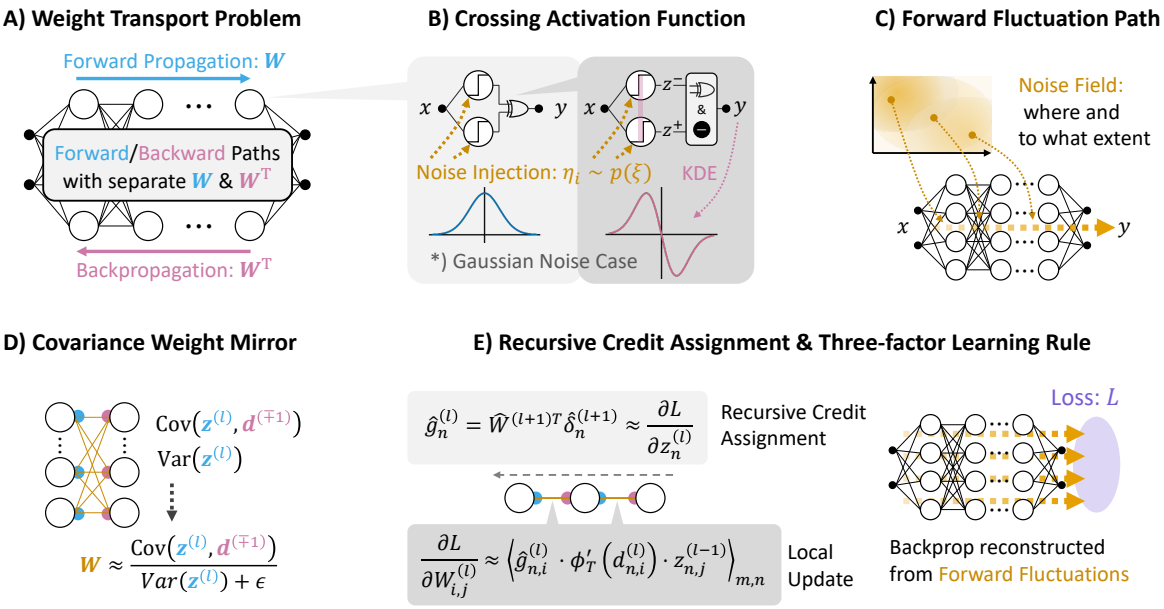

Figure 1. Overview of the problem and proposed approach. (A) Conventional backpropagation requires a backward path and access to transposed forward weights, resulting in the weight transport problem. (B) The crossing activation function generates stochastic binary activity and permits distribution-free estimation of its local derivative by kernel density estimation based on forward noise samples. (C) Noise-induced fluctuations propagate through the network during forward computation. (D) Forward weights are estimated from the covariance between activations and subsequent pre-activations, forming a covariance weight mirror. (E) The estimated weights and local derivatives enable recursive credit assignment and a local three-factor learning rule, thereby reconstructing backpropagation from forward fluctuations.

activation function can be estimated from forward propagation samples with noise. This estimation requires no prior knowledge of the noise probability distribution. Consequently, despite using spike-like binary signals, the NNN can be efficiently trained via backpropagation using estimated gradients rather than surrogate gradients[12]. Thus, the NNN trains via backpropagation despite using spike-like signals.

Although backpropagation makes the NNN practical, it undermines its biological plausibility and neuromorphic validity. Backpropagation requires multiplying the output error by the transposed matrix of the forward weights. This process, called weight transport, requires a separate backward data path, a transpose-accessible weight memory, and strict synchronization between phases. Consequently, backpropagation is difficult to implement in efficient dedicated hardware [13, 14]. It also lacks biological plausibility because it requires centralized execution and lacks a known neuroscientific basis [15, 16]. Various methods have attempted to solve this issue, such as bypassing weight transport, estimating weights, or using local search [17, 18, 19, 20, 3, 21]. However, these methods face challenges, including training instability from inaccurate gradients and reduced model validity due to retrofitted processes. This problem stems from the trade-off between engineering utility and biological plausibility, creating a need for a better compromise.

This study leverages the noise already used in the NNN for learning and inference. We show that the structure of backpropagation can be naturally reconstructed solely from forward-pass statistics by exploiting noise-induced fluctuations. As summarized in Fig. 1, the proposed approach replaces the transported transposed weights with a covariance-based weight mirror and combines it with local derivative estimation to recursively assign credit. Specifically, we build on two related mechanisms: the weight mirror, which estimates interlayer weights from the covariance between the activations of the preceding layer and the preactivation variables of the subsequent layer [20], and the Kolen-Pollack method, which maintains alignment between forward and feedback weights through coordinated parameter updates [19]. By obtaining the required statistics from forward perturbation samples, we integrate the weight mirror without retrofitting dedicated processes. This approach allows us to estimate backward weights and derivatives using only forward computation, enabling recursive error propagation along the computation graph. Furthermore, we show that this model is suitable for hardware implementation. With uniform noise, the entire system can be built using simple digital elements. Uniform random numbers can be generated efficiently using only a few XOR and shift operations, such as Xorshift or linear feedback shift registers (LFSRs), eliminating complex circuits like Box-Muller approximations. Under uniform noise, the crossing

activation reduces to two comparators and an XOR operation, and its local derivative becomes a simple linear expression. Thus, uniform noise allows the entire system, including training and inference, to be simply implemented in digital circuits.

In summary, the contributions of this paper are as follows:

1. **A family of covariance credit assignments for NNN:** We systematically construct a learning rule that recursively propagates errors via a covariance weight mirror, derived from analyzing estimator bias.
2. **Training accuracy comparable to backpropagation:** We verify that the proposed method provides near-unbiased gradient estimates that closely match backpropagation gradients, achieving equivalent final accuracy in a regression task.
3. **High digital hardware friendliness:** We show that uniform noise reduces activation functions and derivatives to simple expressions, enabling hardware implementation using fewer multipliers, efficient random number generators, and standard digital elements.

The remainder of this paper is organized as follows. Section 2 reviews related work, and Section 3 formalizes the NNN and its activation function. Section 4 introduces the forward-only learning rules, followed by experimental validation in Section 5. Section 6 discusses the hardware suitability of the uniform-noise NNN. Section 7 provides an overall discussion, and Section 8 concludes the paper.

2 Related Works

This study relates to three major research fields. This section positions this study by discussing computational models that use noise as a computational resource and two contrasting approaches to the weight transport problem in backpropagation.

2.1 Neuromorphic Computing with Noise as a Resource

Stochastic resonance is the original form of the view that treats noise as a benefit rather than a nuisance. In nonlinear systems, a certain amount of noise enhances the detection and transmission of weak inputs. First discovered in climate dynamics [22, 23], it was later theoretically formalized as a general physical phenomenon [24, 1, 25]. In biological systems, stochastic resonance is widely demonstrated in sensory pathways such as crayfish mechanoreceptors [26], cricket cercal systems [27], and human tactile senses [28], confirming that organisms exploit noise. In the central nervous system, trial-to-trial variability is recognized as a resource for encoding, exploration, and computation rather than mere error [7]. Particularly, computational neuroscience interprets stochastic firing as sampling for probabilistic inference [29, 30, 31, 32]. Here, noise serves to generate samples from a posterior distribution. In machine learning, noise injection is widely used to stabilize and generalize training [33, 34, 35].

Stochastic and spiking neuron models treat noise as a core computational principle rather than an addition to deterministic computation [36, 13]. They generate discrete spikes probabilistically and convey continuous information through expected values. However, their non-differentiable activity is incompatible with gradient-based learning. The standard solution is the surrogate gradient method [37, 38] or the straight-through estimator [39], which replace step-function derivatives with smooth alternatives. Yet, these surrogate derivatives are hand-designed independently of the forward computation. This leaves a fundamental mismatch between inference and learning, known as the surrogate gradient mismatch [12]. Alternatively, escape-noise models feature a firing probability that is a smooth function of the membrane potential. Thus, gradients of expected firing rates or log-likelihoods can be derived for training [40, 41]. However, this requires explicitly assuming the analytical form of the firing probability function. Hardware approaches also leverage randomness, such as stochastic computing, where multiplication is an AND operation and addition is multiplexing over bitstreams [42].

In these research fields, the NNN is unique because it fully utilizes noise to the extent that it functions only through stochastic resonance, requiring no prior knowledge of the noise distribution [9, 10, 11]. Furthermore, because both the forward response and its derivative are derived from the same noise statistics, it avoids the surrogate gradient mismatch. This study uses this property as a starting point. We repurpose the stochastic samples generated for inference to perform credit assignment during learning.

2.2 Forward Learning Rules with True Gradient Convergence

Backpropagation requires transposed weight matrices for hidden-layer credit assignment. This weight transport problem has long challenged both biological plausibility and hardware implementation [16, 15, 43]. Approaches to this problem are categorized by whether they recover the true gradient. This section reviews methods aiming for true gradient recovery. Target propagation [44, 45, 46], equilibrium propagation [47], and predictive coding [48] approximate backpropagation gradients under specific conditions through inverse mappings, equilibrium states, or energy minimization. More directly, the Kolen–Pollack method [19] and weight mirrors [20] align weights with the true transpose. The Kolen–Pollack method asymptotically matches forward and backward weights via identical updates. Weight mirrors directly estimate the transpose using correlations from layer-wise noise injection. Although these methods converge to the true gradient, they require separate, retrofitted processes independent of standard inference and learning. This requirement limits their biological and neuromorphic plausibility. In contrast, the NNN inherently features these perturbations in every forward pass. We exploit this property to construct weight estimation and recursive error propagation solely from forward statistics without any separate process.

2.3 Simple Forward Learning Rules with Deviations from True Gradients

The other approach compromises true gradient convergence to seek local and simple update rules that require neither backward data paths nor weight transport. Representative examples include feedback alignment, which uses a random matrix unrelated to the weight matrix transpose, and direct feedback alignment, which projects output errors directly to each hidden layer [17, 18]. The sign-symmetry method, which shares only the signs of forward weights with the backward matrix, also falls into this category [49, 50]. Methods that estimate gradients from correlations between perturbations and loss changes include node perturbation [51, 3], weight perturbation or SPSA [21], REINFORCE-like policy gradients [52], and forward gradient or zeroth-order methods [53, 54]. Another group optimizes local objective functions using only forward propagation. This includes the Forward-Forward algorithm, contrasting the goodness of positive and negative examples [55], PEPITA utilizing an error-modulated second forward pass [56], and greedy layer-wise learning assigning auxiliary objectives to each layer [57, 58]. For spiking networks, e-prop localizes temporal credit via eligibility traces [59]. All these methods offer high biological and neuromorphic plausibility due to their locality and simplicity. However, they suffer from fundamental issues, such as systematic errors in weight updates, poor convergence, and instability [51, 60]. Consequently, their performance rarely matches that of backpropagation [61, 62]. Our proposed learning rule shares simplicity and locality with these methods by updating weights using only local statistics and treating spontaneous fluctuations of crossing activation as perturbations. Nevertheless, its objective is to reconstruct backpropagation using local statistics, providing a new compromise between Section 2.2 and Section 2.3.

3 Noise-modulated Neural Network

This section formalizes the NNN examined in this study. Section 3.1 introduces the crossing activation function as its basic element, presenting its expected response, noise-induced local derivative, and distribution-free gradient estimation. Section 3.2 defines the network architecture stacked with these functions and its readout via ensemble averaging while introducing a noise field to control the noise of each unit.

3.1 Crossing Activation Function

The basic element of the NNN is the crossing activation function, which fires only in the presence of noise. For an input $d \in \mathbb{R}$, the output $z \in \{0, 1\}$ is defined using independent and identically distributed noise $\eta_1, \eta_2 \overset{\text{i.i.d.}}{\sim} p(\xi)$ as follows:

$$z = \phi(d) = \begin{cases} 1 & \text{if } (d \geq \eta_1) \,\dot{\vee}\, (d \geq \eta_2) \\ 0 & \text{otherwise} \end{cases} \tag{1}$$

where $\dot{\vee}$ denotes the exclusive OR (XOR) operation. Intuitively, this activation function acts as a simple spiking neuron that outputs 1 only when the input crosses the noise. Without noise, where $p(\xi) = \delta(\xi)$, no crossing event occurs for a constant input, making the output identically 0.

The cumulative distribution function of the noise is defined as follows:

$$F(d) = P(d \geq \eta) = \int_{-\infty}^{d} p(\xi)\, d\xi. \tag{2}$$

The expected value of the crossing activation is then given by the following equation.

$$\mathbb{E}[z] = \bar{\phi}(d) = 2F(d)\big(1 - F(d)\big). \tag{3}$$

Because F is monotonically non-decreasing, $\bar{\phi} : \mathbb{R} \to [0, 0.5]$ forms a unimodal, bell-shaped response that peaks at $F(d) = 0.5$, functioning as a smooth basis similar to a radial basis function. Although the binary output z loses most information about the continuous input, the collective firing statistic $\bar{\phi}(d)$ preserves this continuous information through stochastic resonance. Differentiating the expected response $\bar{\phi}$ with respect to d yields the noise-induced local derivative as follows:

$$\bar{\phi}'(d) = 2\big(1 - 2F(d)\big)\, p(d). \tag{4}$$

This indicates that the noise-induced statistical gradient is analytically determined, despite the system consisting of binary step functions. The derivative $\bar{\phi}'$ is positive below the point where $F(d) < 0.5$, negative above it, and vanishes away from the point. As discussed in Section 2.1, while surrogate gradients in spiking networks replace step-function derivatives with hand-designed smooth functions, the NNN derives both the forward response and its derivative from the same noise statistics, eliminating any mismatch between inference and learning [10].

The crossing activation function does not restrict the noise distribution. The definition of ϕ, the expected response $\bar{\phi} = 2F(1 - F)$, and the local derivative $\bar{\phi}' = 2(1 - 2F)\, p$ hold directly for any $p(\xi)$ with a cumulative distribution function F. For example, in the case of Gaussian noise $p(\xi) = \mathcal{N}(0, \sigma^2)$, $F(d) = \frac{1}{2}\big(1 + \operatorname{erf} \frac{d}{\sqrt{2}\,\sigma}\big)$, the expected response and local derivative are given as follows:

$$\bar{\phi}(d) = \frac{1}{2}\left(1 - \operatorname{erf}^2 \frac{d}{\sqrt{2}\,\sigma}\right) \tag{5}$$

$$\bar{\phi}'(d) = -\sqrt{\frac{2}{\pi}}\,\frac{1}{\sigma}\,\operatorname{erf}\left(\frac{d}{\sqrt{2}\,\sigma}\right) e^{-d^2/(2\sigma^2)} \tag{6}$$

where erf denotes the error function. The experiments in this paper primarily use this Gaussian noise.

Although the analytical form of $\bar{\phi}'$ requires prior knowledge of the noise distribution $p(\xi)$, the crossing activation embeds a distribution-free derivative estimator. Consider two crossing outputs $z^{\pm} = \phi(d \pm h)$ shifted by $\pm h$ under the same noise sample, with their respective T-sample averages denoted as $\bar{z}^+$ and $\bar{z}^-$. Using these averages, the estimators for the expected response and its local derivative are obtained as follows:

$$\phi_T(d) = \frac{1}{2}\big(\bar{z}^+ + \bar{z}^-\big) \approx \bar{\phi}(d) \tag{7}$$

$$\phi'_T(d) = \frac{\bar{z}^+ - \bar{z}^-}{2h} \approx \bar{\phi}'(d) \tag{8}$$

The estimator ϕ'_T corresponds to a kernel density estimation using a uniform kernel with bandwidth h, converging to $\bar{\phi}'$ as $T \to \infty$ and $h \to 0$. Statistically, it is crucial that z^+ and z^- are evaluated on the same noise sample. Shifting the threshold by $\pm h$ is equivalent to shifting the pre-activation d by $\mp h$. Therefore, ϕ'_T represents a symmetric finite difference $[z(d+h) - z(d-h)]/(2h)$ under common random numbers, which exhibits low variance because the shared noise cancels out through subtraction. Consequently, the crossing activation embeds a gradient estimator that performs variance reduction of the antithetic or common random numbers type directly on its own samples without additional costs. This slope estimation never references $p(\xi)$, allowing it to operate with the same circuit and the same code for arbitrary noise distributions. The forward learning rules in Section 4 employ this ϕ'_T as the local sensitivity $\partial z/\partial d$.

3.2 Network Structure and Readout

The NNN examined in this study is a fully-connected feedforward network that applies crossing activation element-wise. For an input vector $\mathbf{z}^{(0)} = \mathbf{x}$, the pre-activation and activation of hidden layer $l = 1, \ldots, L-1$ are defined as follows:

$$\mathbf{d}^{(l)} = W^{(l)}\mathbf{z}^{(l-1)} + \mathbf{b}^{(l)} \tag{9}$$

$$\mathbf{z}^{(l)} = \Phi^{(l)}\big(\mathbf{d}^{(l)}\big) \tag{10}$$

where $\Phi^{(l)}$ represents the element-wise application of ϕ. The output layer uses a linear readout as follows:

$$\mathbf{y} = W^{(L)}\mathbf{z}^{(L-1)} + \mathbf{b}^{(L)} \tag{11}$$

Trainable parameters are restricted to $\theta = \{W^{(l)}, \mathbf{b}^{(l)}\}$, and noise is a hyperparameter. Regarding notation, lowercase bold letters denote vectors listing the quantities of all units within a layer, and uppercase $W^{(l)}$ denotes the connection weight matrix.

As detailed in [11], the crossing activation has three implementation levels. These are the sample level (ϕ itself) outputting binary stochastic spikes, the statistical level outputting stochastic continuous values via T-sample averages, and the analytical level deterministically outputting the expected value $\bar{\phi}$. This study focuses on the sample-level network.

In a single inference pass, each unit independently samples noise T times to generate a sequence of binary firing samples $z^{(l)}_{(m)}$ $(m = 1, \ldots, T)$. The network output is the ensemble average of the linear readout, as follows:

$$\bar{\mathbf{y}} = \frac{1}{T} \sum_{m=1}^{T} \mathbf{y}_{(m)} \tag{12}$$

Crucially, these T stochastic forward samples are originally computed for inference. The learning rules in Section 4 reuse them as statistics for credit assignment without requiring additional stochastic forward passes. The sample-wise loss $L_{(m)} = \|\mathbf{y}_{(m)} - \mathbf{t}\|^2$ and the firing fluctuation of each unit $z_{(m)} - \bar{z}$ serve as raw data for the covariance estimation in Section 4.

This formulation clearly shows that the NNN can be trained via backpropagation. The network is a composition of layers with a linear readout, and each crossing activation provides its expected response $\bar{\phi}$ and its local derivative, such as the analytical form $\bar{\phi}'$ or the distribution-free estimator ϕ'_T, during forward propagation. Therefore, standard backpropagation applies directly by propagating the readout error $2(\bar{\mathbf{y}} - \mathbf{t})$ backward along the chain rule, multiplying by $W^{(l)\mathsf{T}}$ and the local derivative at each layer, enabling efficient training of the NNN. Our study starts from this baseline that relies on weight transport via the backward path using the transposed forward weights $W^{(l)\mathsf{T}}$. Section 4 reconstructs these transposed weights and the backward path solely from statistics obtained during forward propagation.

It is noteworthy that the noise distribution does not need to be shared across all units. Letting $p^{(l,k)}$ be the noise distribution of the k-th unit in layer l, the entire set is termed the noise field. The noise field is given by the following expression:

$$\mathcal{P} = \left\{ p^{(l,k)} \right\}_{l=1\ldots L-1,\ k=1\ldots K(l)} \tag{13}$$

where $K(l)$ indicates the number of units in the l-th layer. Units with $p^{(l,k)} = \delta$ (no noise) produce outputs and gradients that are identically 0, completely disconnecting them from both inference and learning. Thus, the noise field acts as a hyperparameter that determines which sub-network is recruited for computation. This study employs a minimal configuration where noise intensity (scale parameter, such as σ in a Gaussian distribution) in each unit is modulated by a scalar field $s_k \in [0, 1]$. The learning rules in Section 4 naturally align with this field, meaning units with $s_k = 0$ receive a covariance credit of exactly 0 and are automatically excluded from updates. This implies that credit assignment and recruitment share the same field, which has implications for the power gating discussed in Section 7.

4 Forward Fluctuation Covariance Learning

The NNN in Section 3 generates multiple stochastic forward samples for each input, including layer-wise pre-activations, activations, readout outputs, and sample-wise losses. The learning rules proposed in this section estimate the gradients for weight updates using only these forward quantities. They require no transposed weight matrices or autograd. Because optimizers such as SGD and Adam operate locally without causing weight transport, estimating gradients from forward statistics is the core requirement to solve the weight transport problem. Following the problem setup in Section 4.1, we sequentially introduce the simplest scalar covariance rule in Section 4.2, the structured covariance rule in Section 4.3 to overcome its limitations, and the generalization in Section 4.4 to replace the remaining analytical loss derivatives.

4.1 Problem Setup

For a dataset $\mathcal{D} = \{\mathbf{x}_n, \mathbf{t}_n\}_{n=1}^N$, the NNN in Section 3.2 generates T stochastic forward samples per input. The network consists of layers $l = 1, \ldots, L$, where layer L is a linear readout. Let $\mathbf{d}^{(l)}_{(m,n)}$ and $\mathbf{z}^{(l)}_{(m,n)}$ be the pre-activation and activation of layer l for input n and sample m, $\mathbf{y}_{(m,n)}$ be the readout output, and $L_{(m,n)} = \|\mathbf{y}_{(m,n)} - \mathbf{t}_n\|^2$ be the sample-wise loss. Subscripts in parentheses (m, n) denote the sample and input indices, whereas subscripts without parentheses i and j

indicate vector elements or units. For example, $z^{(l)}_{(m,n),i}$ is the i-th element of $\mathbf{z}^{(l)}_{(m,n)}$, representing the activation of unit i in layer l. Regarding notation, an overbar denotes a T-sample average for a fixed input n, and omitting the sample index m refers to an observation sequence over the same T samples. The operator $\langle\cdot\rangle_{m,n}$ represents the average over all samples and inputs, and ϵ is a small positive constant to prevent division by zero. The network output is the ensemble average $\bar{\mathbf{y}}_n$. The training objective function L is the average squared error $\|\bar{\mathbf{y}}_n - \mathbf{t}_n\|^2$ over all inputs, which is contextually distinguished from the layer index L. The readout error signal is given by $\mathbf{e}_n = 2(\bar{\mathbf{y}}_n - \mathbf{t}_n)$.

Because $\bar{\mathbf{y}}_n$ is linear with respect to $W^{(L)}$, the readout gradient can be expressed exactly and locally using the ensemble-averaged activation $\bar{\mathbf{z}}^{(L-1)}_n$ as follows:

$$\frac{\partial L}{\partial W^{(L)}} = \frac{1}{N}\sum_n \mathbf{e}_n\, \bar{\mathbf{z}}^{(L-1)\mathsf{T}}_n \tag{14}$$

Therefore, eliminating the direct access to the transposed forward weights reduces entirely to the hidden-layer credit assignment problem, which requires estimating $\partial L/\partial z^{(l)}_i$ solely from forward statistics.

4.2 Scalar Covariance Credit Rule

The simplest configuration is the regression of the loss on the spontaneous fluctuations of each unit. Using the covariance centered over T samples for each input n, the credit for the activation is defined as follows:

$$g^{(l)}_{n,i} = \frac{\mathrm{Cov}_T\big(L_n,\, z^{(l)}_{n,i}\big)}{\mathrm{Var}_T\big(z^{(l)}_{n,i}\big) + \epsilon} \;\approx\; \frac{\partial L_n}{\partial z^{(l)}_{n,i}}. \tag{15}$$

Here, Cov_T and Var_T are the empirical covariance and variance over T samples for a fixed input n. These terms are calculated from only four sample averages accumulated during the forward pass, specifically $\overline{L}_n$, $\overline{z}_{n,i}$, $\overline{Lz}_{n,i}$, and $\overline{z^2}_{n,i}$, which represent the averages of L, z_i, Lz_i, and z_i^2, respectively, with the layer index omitted as follows:

$$\mathrm{Cov}_T\big(L_n,\, z^{(l)}_{n,i}\big) \;=\; \overline{Lz}_{n,i} - \overline{L}_n\,\overline{z}_{n,i} \tag{16}$$

$$\mathrm{Var}_T\big(z^{(l)}_{n,i}\big) \;=\; \overline{z^2}_{n,i} - \overline{z}^2_{n,i}. \tag{17}$$

This credit replaces the $W^{(l+1)\mathsf{T}}\boldsymbol{\delta}^{(l+1)}$ term of backpropagation, where $\boldsymbol{\delta}^{(l+1)}$ is the error signal propagated from higher layers, with forward statistics.

Multiplying this credit by the distribution-free local slope ϕ'_T from Section 3.1 constructs a pseudo-error, yielding the hidden-layer weight gradient estimate as follows:

$$\hat{\delta}^{(l)}_{n,i} \;=\; g^{(l)}_{n,i}\,\phi'_T\big(d^{(l)}_{n,i}\big) \tag{18}$$

$$\hat{G}^{(l)} \;=\; \Big\langle \hat{\boldsymbol{\delta}}^{(l)}\mathbf{z}^{(l-1)\mathsf{T}} \Big\rangle_{m,n} \;\approx\; \frac{\partial L}{\partial W^{(l)}} \tag{19}$$

Because $g\cdot\phi'_T \approx (\partial L/\partial z)(\partial z/\partial d) = \partial L/\partial d$, the gradient estimate takes the same outer-product form $\boldsymbol{\delta}\mathbf{z}^{\mathsf{T}}$ as backpropagation. This rule, which forms the pseudo-error $\hat{\boldsymbol{\delta}}$ by multiplying the scalar covariance credit g by the local slope ϕ'_T and computes $\hat{G}^{(l)}$ via an outer product, is referred to as "`cov_deriv`" in this paper.

The `cov_deriv` rule can be interpreted as deeply integrating node perturbation into the NNN. Classical node perturbation injects an independent perturbation ξ into each unit solely for training and regresses the loss change on ξ. In contrast, because crossing activation already fluctuates due to the forward noise, the spontaneous fluctuation $z - \bar{z}$ itself acts as the perturbation, eliminating the need for an additional dedicated process. As a related simple configuration, a minimal variant "`cov_only`" can be considered, which omits the local slope ϕ'_T and directly uses the credit for the gradient estimate. These covariance statistics require only the four accumulations per unit ($\overline{L}$, $\overline{z}$, $\overline{Lz}$, and $\overline{z^2}$), while `cov_deriv` additionally employs a crossing counter for ϕ'_T. Both configurations allow online accumulation as samples are obtained.

However, both `cov_only` and `cov_deriv` face fundamental concerns compared to backpropagation. First, `cov_only` lacks the $\partial z/\partial d$ information as a trade-off for its simplification, which is expected to distort the update direction. Second, the credit estimator of `cov_deriv` retains a systematic error inherent in its structure. The ratio $\mathrm{Cov}(L, z_i)/\mathrm{Var}(z_i)$ is a univariate regression that compresses the downstream network effects of unit i into the scalar loss L,

linearizing and ignoring interactions when multiple units fluctuate simultaneously. Because this linearization error is not a finite-sample effect, it does not vanish by increasing T. Avoiding this compression requires computing the covariance with quantities aligned with the computational graph structure rather than the scalar L.

4.3 Structured Covariance Credit Rule

4.3.1 Bypassing the Weight Transport Problem Pre-activations are strictly linear with respect to activations ($\mathbf{d}^{(l+1)} = W^{(l+1)}\mathbf{z}^{(l)} + \mathbf{b}^{(l+1)}$). Therefore, the pooled single-regression coefficients of the T-sample fluctuations for all inputs, when the input is fixed, yield the forward weights themselves as follows:

$$\hat{W}_{ji}^{(l+1)} = \frac{\sum_n \mathrm{Cov}_T\left(d_{n,j}^{(l+1)},\, z_{n,i}^{(l)}\right)}{\sum_n \mathrm{Var}_T\left(z_{n,i}^{(l)}\right) + \epsilon} \;\approx\; W_{ji}^{(l+1)} \tag{20}$$

This approach adapts the weight mirror method, which estimates the transposed weight matrix from input-output correlations under noise injection [20]. The key difference is that instead of requiring a dedicated noise injection phase, this approach exploits the fluctuations already generated by the NNN during the forward pass. In addition to the accumulated quantities in Section 4.2, the system stores $\overline{d}_{n,j}$ for each unit and the average product $\overline{dz}_{n,ji}$ for each unit pair (j, i). The numerator is given by $\mathrm{Cov}_T\left(d_{n,j}^{(l+1)},\, z_{n,i}^{(l)}\right) = \overline{dz}_{n,ji} - \overline{d}_{n,j}\,\overline{z}_{n,i}$, and the denominator is computed from $\overline{z}$ and $\overline{z^2}$ shared with Section 4.2. Because the crossing noise of each unit fluctuates independently for a fixed input, this regression largely suppresses confounding and enables highly accurate estimation.

The following three points are key implementation requirements to improve estimation efficiency and accuracy:

1. The correlation is computed with the pre-activation $d^{(l+1)}$. Since $d_j^{(l+1)}$ is the weighted sum of the previous layer firing and is strictly linear with respect to $\mathbf{z}^{(l)}$, the regression coefficient directly provides W_{ji}.

2. Unlike gradients, weights do not depend on the input, so first sum the numerator and denominator over all inputs and then divide. This allows all NT samples included in a single parameter update step to be used in a single estimation (resulting in lower variance than calculating the ratio for each input and then averaging it).

3. The estimation $\hat{W}$ is performed at each update step using an exponential moving average with a smoothing constant of 0.9. Weight changes are incorporated separately via $\hat{W} \leftarrow \hat{W} + \Delta W$, adapting the Kolen-Pollack method [19].

4.3.2 Recursive Error Propagation Using $\hat{W}$ and ϕ'_T estimated from forward perturbation samples, the output error can be propagated through a recursion isomorphic to backpropagation.

$$\hat{\boldsymbol{\delta}}_n^{(L-1)} = \left(\hat{W}^{(L)\mathsf{T}}\,\mathbf{e}_n\right) \odot \phi'_T\left(\mathbf{d}_n^{(L-1)}\right) \tag{21}$$

$$\hat{\boldsymbol{\delta}}_n^{(l)} = \left(\hat{W}^{(l+1)\mathsf{T}}\hat{\boldsymbol{\delta}}_n^{(l+1)}\right) \odot \phi'_T\left(\mathbf{d}_n^{(l)}\right) \tag{22}$$

The gradient estimate is obtained via the outer product $\hat{G}^{(l)} = \langle \hat{\boldsymbol{\delta}}^{(l)} \mathbf{z}^{(l-1)\mathsf{T}} \rangle_{m,n}$ as in `cov_deriv`. Compared to backpropagation, the only difference is that W^{T} is replaced by $\hat{W}^{\mathsf{T}}$ estimated from forward perturbations.

The inter-layer Jacobian multiplied in backpropagation via the chain rule consists of two factors, namely the linear combination part $\partial \mathbf{d}^{(l+1)}/\partial \mathbf{z}^{(l)} = W^{(l+1)}$ and the local derivative of the activation $\partial z/\partial d$. The recursion above replaces these two factors with $\hat{W}$ and ϕ'_T estimated from forward statistics. This represents a reconstruction of backpropagation based on Jacobian estimation via forward perturbations. Hereafter, this method is referred to as "`cov_jac`."

Unlike `cov_deriv`, `cov_jac` propagates the credit along the computation graph without compressing it into the scalar loss L. This eliminates linearization bias and allows the estimation error to decrease as the number of samples increases. For a layer width of K units, the statistics require the pairwise average product $\overline{dz}$, which increases the computational complexity from $O(K)$ in `cov_deriv` to $O(K^2)$ in `cov_jac`. Nevertheless, it remains forward-only and supports online accumulation. The simple regression for $\hat{W}$ is a diagonal approximation assuming uncorrelated activations within a layer. Strong intra-layer correlations could cause weight leakage from

neighboring units, creating a directional bias that persists even with larger sample sizes. However, this effect is negligible in our regime because the unique crossing noise of each unit dominates.

4.4 Generalized Structured Covariance Credit Rule

Up to this point, the only analytically computed quantity is the readout error signal $\mathbf{e}_n = 2(\bar{\mathbf{y}}_n - \mathbf{t}_n)$. While this term is independent of weight transport, this final loss derivative can also be replaced with forward statistics. In the following, each readout unit is treated element-wise, omitting element subscripts to write y, t, and e. Because the sample output $y_{(m,n)}$ is a continuous quantity that fluctuates with forward perturbations, the same per-input regression used for hidden layers can be applied as follows:

$$g_{y,n} = \frac{\mathrm{Cov}_T(L_n,\, y_n)}{\mathrm{Var}_T(y_n) + \epsilon} \;\approx\; e_n. \tag{23}$$

The only newly accumulated quantities are $\overline{y}_n$, $\overline{Ly}_n$, and $\overline{y^2}_n$ for each readout unit, while $\overline{L}_n$ is shared with Section 4.2. In this paper, the rule that uses this $g_{y,n}$ both as the starting point for recursion to seed $\hat{\boldsymbol{\delta}}^{(L-1)}$ and for estimating the readout weight gradient is called `cov_jac_full`. In this configuration, the network observes only the scalar loss per sample, meaning no analytical loss derivative appears anywhere.

However, this regression introduces a systematic error. Since the loss $L = (y-t)^2$ is a quadratic function of y, the population regression coefficient is strictly given by the following expression:

$$\frac{\mathrm{Cov}(L, y)}{\mathrm{Var}(y)} \;=\; 2\big(\mathbb{E}[y] - t\big) + \frac{\mathbb{E}[\varepsilon^3]}{\mathrm{Var}(\varepsilon)} \tag{24}$$

$$\varepsilon \;=\; y - \mathbb{E}[y]. \tag{25}$$

The second term is a constant bias proportional to the skewness of the readout fluctuations. Because y is a weighted sum of binary firings, this term is generally non-zero and persists even with larger sample sizes T. This bias is less prominent in SGD because the step size shrinks along with the gradient magnitude. However, Adam continually amplifies this bias through gradient scale normalization, which can cause the solution to drift in the later stages of training.

Two correction methods can eliminate the need for analytical loss derivatives. The third central moment correction subtracts the third central moment of y, given by $m_3 = \overline{y^3}_n - 3\,\overline{y}_n\,\overline{y^2}_n + 2\,\overline{y}_n^3$, from the numerator, where $g_y = (\mathrm{Cov} - m_3)/\mathrm{Var}$. The symmetric probe adds a known symmetric noise ξ satisfying $\mathbb{E}[\xi^3] = 0$ to the readout samples and regresses the loss on ξ instead of y, where $g_y = \mathrm{Cov}_T(L(y+\xi), \xi)/\mathrm{Var}_T(\xi)$. These two methods form a complementary pair. The third central moment correction requires one additional accumulation $\overline{y^3}_n$ and achieves exact bias removal only for quadratic losses. Nevertheless, because it operates solely on the observed quantities (L, y), it requires no additional perturbation injection and integrates naturally into the NNN. In contrast, the symmetric probe requires no additional accumulation and remains closely aligned for any smooth loss function by accumulating identical forms of $\overline{\xi}$, $\overline{L\xi}$, and $\overline{\xi^2}$ instead of $\overline{y}$, $\overline{Ly}$, and $\overline{y^2}$. However, it requires injecting an additional perturbation, which partially undermines our claim that the method integrates naturally into the NNN. This study uses the third central moment correction as the default configuration.

5 Experimental Results

5.1 Experimental Setup

This section evaluates a simple regression task approximating $\sin(x)$. The network architecture is 1–64–64–1 with an ensemble size of $T = 64$, and training is performed for 1500 epochs using identical initial weights across all configurations. The evaluation compares five methods, namely backprop, `cov_only`, `cov_deriv`, `cov_jac`, and `cov_jac_full`. The four covariance-based learning rules introduced in Section 4 operate without autograd, updating weights manually under the no_grad context. The credit and computational requirements for each method are summarized in Table 1.

Section 4 describes the hidden-layer credit and readout error, allowing parameter updates with any optimizer. In addition to SGD, the first and second moments m and v of Adam are local to each weight, ensuring that Adam causes no weight transport. However, the suitability of the optimizer depends on the error properties of the estimator. The $1/\sqrt{v}$ normalization of Adam standardizes the gradient scale. When applied to `cov_only` or `cov_deriv`, which have the persistent systematic errors described in Section 4.2, this normalization continuously amplifies the bias

Table 1. Comparison of the learning signals and computational requirements of the evaluated methods. The table lists the hidden-layer credit, readout-error signal, optimizer, asymptotic additional memory per layer, additional computation per parameter update, and the requirement for a backward path or transposed-weight access. Here, H denotes the hidden-layer width and T denotes the number of stochastic forward samples. The corresponding learning rules are detailed in Section 4.

Method	backprop	`cov_only`	`cov_deriv`	`cov_jac`	`cov_jac_full`
Hidden Credit	$W^\top \boldsymbol{\delta}$	$\frac{\mathrm{Cov}(L,z)}{\mathrm{Var}(z)}$	$\frac{\mathrm{Cov}(L,z)}{\mathrm{Var}(z)}\phi'_T$	$\hat{W}^\top \hat{\boldsymbol{\delta}}$	$\hat{W}^\top \hat{\boldsymbol{\delta}}$
Readout Error	$2(\bar{y}-t)$	$2(\bar{y}-t)$	$2(\bar{y}-t)$	$2(\bar{y}-t)$	$\frac{\mathrm{Cov}(L,y)-m_3}{\mathrm{Var}(y)}$
Optimizer	Adam	SGD	SGD	Adam	Adam
Extra Memory per Layer	$O(H^2)$	$O(H)$	$O(H)$	$O(H^2)$	$O(H^2)$
Extra Compute per Update	$O(H^2)$	$O(TH)$	$O(TH)$	$O(TH^2)$	$O(TH^2)$
Access to Transposed Weights	Required	Not required	Not required	Not required	Not required

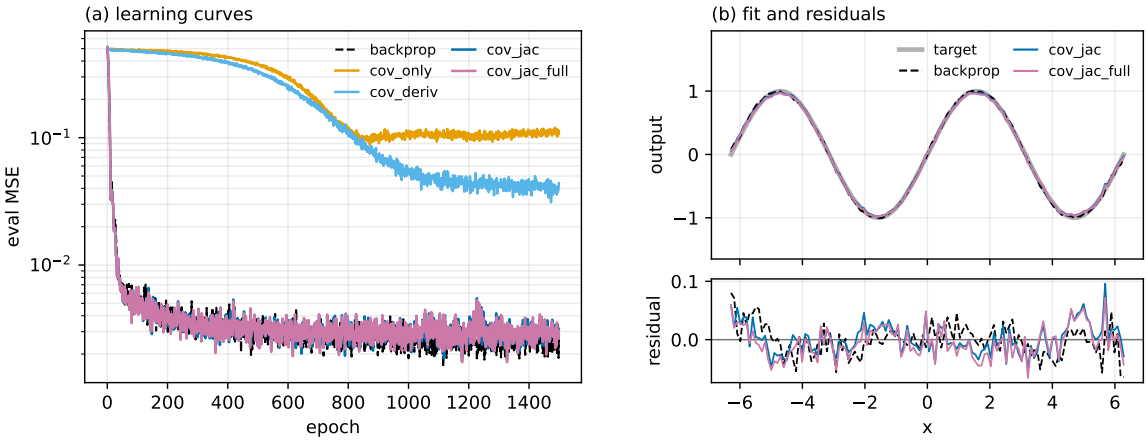

Figure 2. Learning performance on the sin(x) regression task using backprop and proposed method variants. (a) Evaluation MSE over 1500 training epochs for backprop, `cov_only`, `cov_deriv`, `cov_jac`, and `cov_jac_full`. (b) Target function, learned outputs, and residuals after training for backprop, `cov_jac`, and `cov_jac_full`.

component as the true gradient shrinks in later training stages, pushing the solution away from the optimum. In contrast, Adam operates effectively for the low-variance, quasi-unbiased gradients of `cov_jac` and `cov_jac_full` as in backpropagation. Therefore, the main evaluation adopts the specific optimizer suited for each learning rule, as shown in Table 1.

5.2 Main Results

Table 2 lists the final MSE (mean ± std over seeds 0–2) for each method, and Fig. 2 shows the training curves and post-training responses for the first seed. The final MSE values of `cov_jac` and `cov_jac_full` matched backprop within the seed-to-seed variability. In contrast, `cov_only` and `cov_deriv` using scalar covariance credit were limited to 0.096 and 0.035, respectively. This residual error aligns with the persistent systematic errors described in Section 4.2. As shown in Fig. 2(a), the training curves of `cov_jac` and `cov_jac_full` closely overlap with backprop throughout all epochs, showing no difference in convergence speed. In Fig. 2(b), the predictions of both methods overlap with the target function, and the residuals have amplitudes comparable to backprop without exhibiting systematic structures. These results demonstrate that structuring credits via mirror recursion (Section 4.3) achieves a learning performance comparable to backpropagation without transporting or transposing weights. Furthermore, because `cov_jac_full` avoids analytical loss derivatives, all signals required for training are derived from forward sampling statistics.

Table 2. Final evaluation MSE on the $\sin(x)$ regression task after 1500 training epochs. Values are reported as the mean $\pm$ standard deviation over three different seeds for the 1–64–64–1 network with $T = 64$ stochastic forward samples per input.

Method	Final MSE
backprop	0.00057 ± 0.00010
`cov_only`	0.09583 ± 0.00724
`cov_deriv`	0.03464 ± 0.00558
`cov_jac`	0.00056 ± 0.00006
`cov_jac_full`	0.00057 ± 0.00009

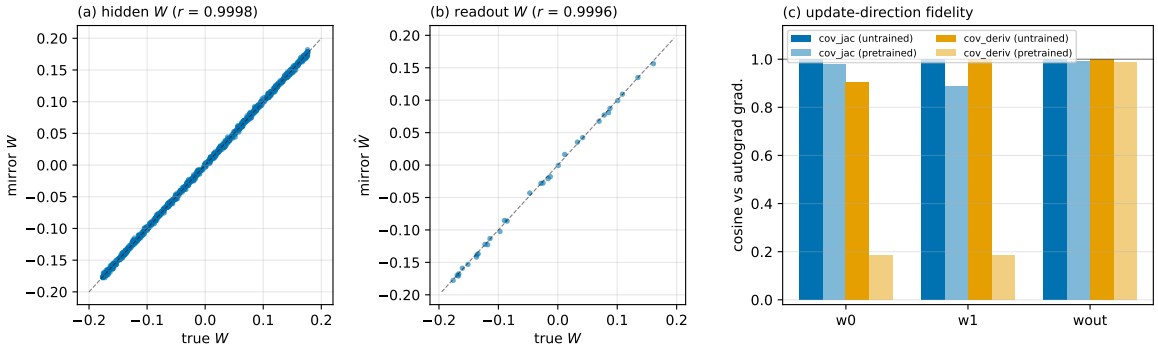

Figure 3. Fidelity of the reconstructed weights and gradient directions. (a, b) Recovered mirror weights $\hat{W}$ versus the true forward weights W for the hidden and readout layers, respectively, in the untrained network; r denotes the Pearson correlation coefficient. (c) Cosine similarity between the layer-wise update directions produced by `cov_jac` or `cov_deriv` and the exact autograd gradients at initialization and after 300 epochs of backpropagation pretraining. Here, "w0" and "w1" denote the two hidden-layer weight matrices, and "wout" denotes the readout weight matrix.

5.3 Verification of the Accuracy of Gradient Reconstruction

To confirm that the agreement in Section 5.2 is not accidental, we directly compare the learning signals of `cov_jac` with exact reference quantities (Fig. 3). First, we evaluate the recovery accuracy of the weight mirror (See Section 4.3), which forms the core of the recursion. Figs. 3(a) and (b) show the scatter plots of the recovered $\hat{W}$ and the true forward weights W in the untrained state, where the values align on the same straight line for both the hidden and readout layers. The Pearson correlation reaches $r \geq 0.999$ in the hidden layers and $r \geq 0.988$ in the readout layer in both untrained and partially trained states after 300 epochs of backpropagation. Next, we compare the layer-wise updates constructed by `cov_jac` with the exact gradients from backpropagation via autograd (Fig. 3(c)). In the untrained initial state, the cosine similarity is 0.998 to 1.000 across all layers, and the norm ratio is 0.99 to 1.00, demonstrating that `cov_jac` reconstructs the gradient almost exactly in both direction and scale. Even during training, the directional agreement is maintained with a cosine similarity of 0.89 to 0.99, although the norm ratio expands up to 2.2. This scale discrepancy is absorbed by the $1/\sqrt{v}$ normalization of Adam, meaning it does not appear in the final performance (Section 5.2). In contrast, the update direction of `cov_deriv` drops to a cosine similarity of 0.19 in the hidden layers during training (Fig. 3(c)). The performance gap between `cov_jac` and `cov_deriv` observed in Section 5.2 is directly explained by this difference in gradient fidelity.

5.4 Ablation Study

The main results in Section 5.2 rely on a combination of design choices, including credit structuring, per-input credit estimation, distribution-free slope estimation, and an estimator-suited optimizer. This section isolates which choices are essential for performance and which are interchangeable. Specifically, we alter four main design axes one by one while maintaining the protocol of Section 5.1. These are (1) optimizer (SGD or Adam), (2) credit estimation unit (per-input or pooled), (3) local slope estimation method (estimated slope ϕ'_T or analytical derivative $\bar{\phi}'$), and (4) presence of the local slope (`cov_deriv` or `cov_only`). In addition, we examine the effect of explicitly tracking parameter updates in the covariance weight mirror. The results are summarized in Fig. 4.

5.4.1 Optimizer When `cov_jac` is trained with SGD, the final MSE is limited to 0.015, whereas it reaches 0.00056 with Adam, as shown by the `cov_jac_sgd/track` and `cov_jac_adam/track`

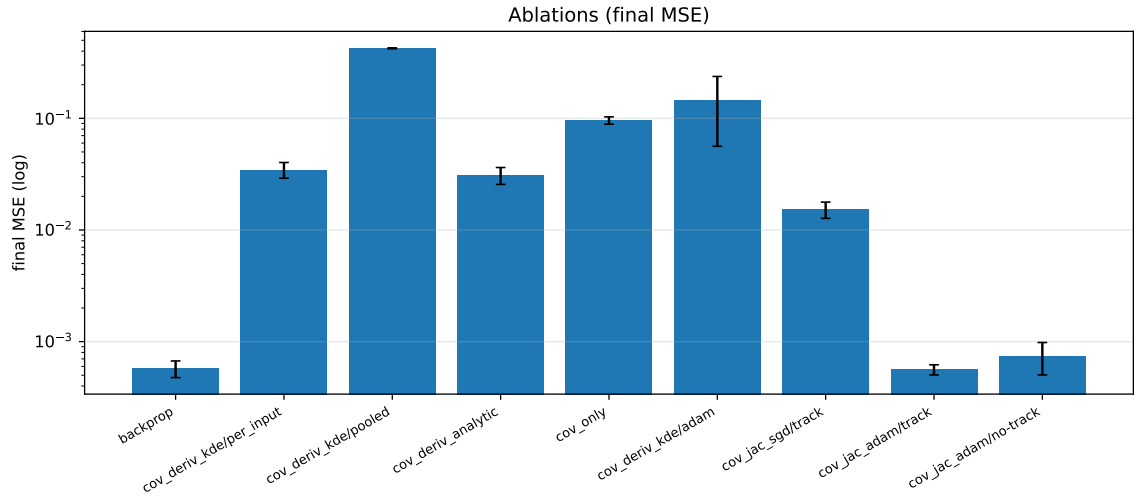

Figure 4. Ablation study of the proposed learning rules. Bars show the final evaluation MSE on a logarithmic scale, with the mean and standard deviation evaluated over three different seeds. The comparisons examine the effects of optimizer selection, per-input versus pooled covariance estimation, distribution-free versus analytical local-slope estimation, omission of the local slope, and explicit tracking of parameter updates in the covariance weight mirror. Backpropagation is included as a reference.

conditions in Fig. 4, respectively. This residual error is an artifact of optimization rather than a bias in the credit estimator. Conversely, switching `cov_deriv` to Adam degrades the final MSE from 0.035 to 0.147 ± 0.091, and the seed-to-seed variability becomes the largest among all comparisons. This indicates that the systematic error described in Section 4.2 is not eliminated by the optimizer, and Adam, which normalizes the gradient scale, amplifies it instead. This asymmetry justifies the choice of optimizers in Section 5.1, and the amplification mechanism is directly demonstrated for the readout estimator in Section 5.5. Removing the explicit mirror-tracking update caused only a small degradation under the present conditions, as shown by the `cov_jac_adam/no-track` condition in Fig. 4.

5.4.2 Credit estimation unit Replacing the per-input credit, which computes covariance statistics for each input, with pooled credit across all inputs degrades the final MSE from 0.035 to 0.42. This represents the largest degradation in Fig. 4, and learning practically fails. Because the covariance between loss and activity approximates the local gradient only when conditioned on the input (Section 4.2), per-input estimation is critical.

5.4.3 Local slope estimation method Replacing the distribution-free estimated slope (Section 3.1) with the analytical $\bar{\phi}'$ keeps the final MSE nearly unchanged at 0.031 compared to 0.035 (Fig. 4). Therefore, the two are interchangeable, and an analytical model of the noise distribution is unnecessary for learning.

5.4.4 Presence of local slope The difference between `cov_only` without $\bar{\phi}'$ (0.096) and `cov_deriv` with it (0.035) illustrates the contribution of multiplying the covariance credit by the local slope (Fig. 4). In conclusion, performance is determined by credit structuring (Section 5.2), per-input estimation, and an optimizer suited to the error properties, while the implementation of slope estimation is interchangeable.

5.5 Empirical Study on the Estimation of the Covariance of Readout Errors
We evaluate the predicted skewness bias and its corrections by replacing the readout error estimation in `cov_jac_full` with the three variants from Section 4.4, which are uncorrected regression, third order moment correction, and the symmetric probe. All other conditions match the main comparison. The final MSE values as mean ± std over seeds 0, 1, and 2 were 0.00057 ± 0.00009 for the third order moment correction, matching backprop at 0.00057 ± 0.00010. The symmetric probe yielded 0.00090 ± 0.00011, and the uncorrected variant was 0.0241 ± 0.0007. Until around epoch 200, the uncorrected curve converges to approximately 0.006 at the same rate as the third order moment correction, but it drifts to 0.024 after epoch 500 (Fig. 5(a)). This

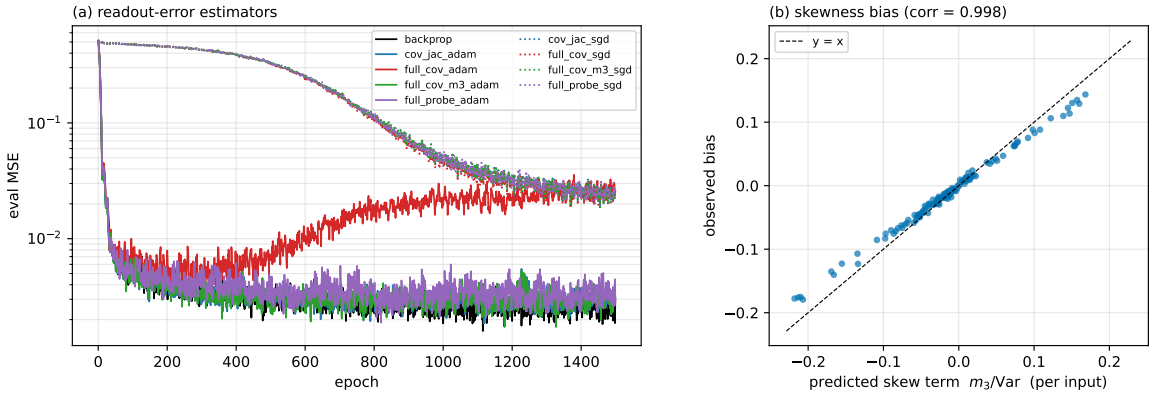

Figure 5. Readout-error estimation from forward covariances and verification of the skewness bias. (a) Evaluation MSE for `cov_jac` and three `cov_jac_full` readout-error estimators: uncorrected covariance regression (`full_cov`), third central moment correction (`full_cov_m3`), and the symmetric probe (`full_probe`). Results using Adam and SGD are compared with backpropagation. (b) Observed readout-error bias versus the predicted skewness term $m_3/\mathrm{Var}(y)$ for individual inputs. The dashed line denotes equality, and the Pearson correlation coefficient is 0.998.

Table 3. Final performance on three benchmark tasks. Entries report the mean ± standard deviation of the final MSE over three different seeds for Friedman #1 regression, Two Moons classification, and Concentric Circles classification. Friedman #1 targets were normalized to $[-1, 1]$, and the classification tasks used labels of ± 1. Classification accuracies are shown in parentheses.

Method	Friedman #1	Two Moons	Concentric Circles
backprop	0.00030 ± 0.00001	0.00035 ± 0.00005 (1.000)	0.00202 ± 0.00053 (1.000)
`cov_only`	0.12043 ± 0.00920	0.80647 ± 0.02375 (0.854)	0.71999 ± 0.03417 (0.943)
`cov_deriv`	0.07735 ± 0.01048	0.13931 ± 0.01394 (0.971)	0.21531 ± 0.00663 (1.000)
`cov_jac`	0.00033 ± 0.00006	0.00044 ± 0.00006 (1.000)	0.00242 ± 0.00068 (1.000)
`cov_jac_full`	0.00033 ± 0.00006	0.00040 ± 0.00015 (1.000)	0.00267 ± 0.00041 (1.000)

behavior confirms the prediction in Section 4.4 that the skewness bias remains after convergence and that Adam continuously amplifies it relatively via gradient scale normalization. Indeed, when using SGD, all three estimators stay at the minimal optimization loss described in Section 5.4, showing no performance gap. We also verified the bias term directly. The observed bias matches the predicted m_3/Var with a correlation of 0.998 for each input. Near convergence, the bias RMS of 0.062 exceeds the true error signal RMS of 0.049 (Fig. 5(b)). Thus, the skewness bias is real and matches the predicted magnitude, and the third order moment correction alone raises the readout covariance estimation to the backpropagation-level.

5.6 Benchmarking

To confirm that the main results are not task specific, we compare the five methods by changing only the tasks while maintaining the identical protocol from Section 5.1 including network architecture, T, epoch count, and optimizer selection as outlined in Table 1. The three tasks are Friedman #1 regression with a five dimensional input and targets normalized from -1 to 1, and binary classification of two moons and concentric circles using MSE learning with targets of ± 1. The two classification tasks are linearly inseparable and cannot be solved unless credit assignment to the hidden layers functions properly. The datasets are fixed, and seeds vary only the initial weights and the noise during training. Table 3 presents the final MSE as mean ± std over seeds 0, 1, and 2, with classification accuracies in parentheses.

As shown in Table 3, the final MSE of `cov_jac` and `cov_jac_full` matches backprop within the range of variability between seeds across all three tasks, and all classification accuracies reach 1.000. The performance ranking among the methods is identical to Section 5.2, where `cov_deriv` achieves intermediate results and `cov_only` ranks lowest. On the two moons task, `cov_only` even fails classification with an accuracy of 0.854. Therefore, the conclusions in Section 5.2 are not unique to $\sin(x)$.

Table 4. Estimated digital-hardware resources per parameter update for backprop, `cov_deriv`, and `cov_jac`. The estimates assume $N = 128$ inputs, $T = 64$ stochastic samples per input, a hidden-layer width of $H = 64$, and two hidden layers. The T forward passes common to all methods are excluded. Binary-gated MACs exploit binary hidden activations and can be implemented using gating and accumulation rather than full-precision multiplication. Memory requirements are reported in words. These estimates characterize architectural requirements and do not represent measured area, power, or latency.

Method	Full-Precision MACs	Binary-Gated MACs	Divisions	Memory (words)	Backward Weight Path
backprop	34.1M	34.6M	0	1.05M + 4.2k (W^T)	Required
`cov_deriv`	0	37.2M	16k	0.5k words	Not required
`cov_jac`	0.53M	69.2M	4.2k	8.4k (two sets)	Not required

6 Hardware-oriented Analysis

6.1 Resource Requirements

This section estimates the extra computation and memory required by the forward learning rules in Section 4 and compares them with backprop. The comparison is limited to a rough estimate at the MAC level, indicating the types and scales of operations without addressing implementation dependent metrics such as power or area. The statistics required for learning are four types of running sums per unit given by $\overline{L}$, $\overline{z}$, $\overline{Lz}$, and $\overline{z^2}$, along with $\mathrm{Cov}(d, z)$ for the mirror. When the readout error is also estimated from the covariance, as in Section 4.4, the addition is limited to one accumulation per readout unit given by $\overline{y^3}$. The required operations are restricted to accumulation, multiplication, subtraction, one division per unit, and MAC, completely eliminating the need for transposed weight memory, a second data path for backward propagation, and phase synchronization between the forward and backward passes.

Table 4 shows a rough estimate of the required extra computation and memory per weight update, excluding the forward pass with $N = 128$, $T = 64$, $H = 64$, and two hidden layers, because the T times forward sampling is common to all methods. Because the hidden activation z is binary, the multiply accumulate operation multiplying z can be implemented with an adder with an AND gate as a binary gate multiply accumulate operation instead of a multiplier, which is distinguished from a normal MAC in the table.

In terms of total operations, `cov_jac` at 69.7M is almost identical to backprop at 68.7M, meaning this method does not excel in computational volume. The difference lies in the breakdown of operations and memory. While half of the extra computation in backprop at 34.1M represents true MAC operations passing through the transposed weight path, `cov_jac` requires multipliers for only 0.53M operations for the credit recursion $\hat{W}^\mathsf{T}\hat{\boldsymbol{\delta}}$, which occurs once per input regardless of T. The remainder consists of binary gate multiply accumulate operations and a small number of divisions. Regarding working memory, while backprop requires a $\boldsymbol{\delta}$ buffer of 1.05M words and a weight duplicate accessible via transposition, `cov_deriv` requires only 0.5k words, and `cov_jac` requires only 8.4k words for the mirror and its exponential moving average. This comparison assumes all methods run the same stochastic model and share the T forward passes, but when compared with backprop on a deterministic network, this T times forward overhead becomes the substantial extra cost of this method. That is, this method replaces the backward infrastructure, including transposed weight memory, a second data path, and synchronization with the number of forward trials. Because whether this replacement is advantageous in power or area depends on the circuits and processes, this paper makes no claims of quantitative superiority and only shows the differences in architectural requirements in Table 4.

There are also limitations to locality. The weight mirror recovery $\hat{W} = \mathrm{Cov}(d, z)/\mathrm{Var}(z)$ is a univariate regression per unit, representing a diagonal approximation that ignores intra-layer activity correlations. Strong intra-layer correlations can cause systematic shifts in the recovered $\hat{W}$, though this effect is small in our regime, with Pearson correlations of $r \geq 0.988$ in the evaluated states, as reported in Section 5.3. The exact multivariate regression given by $\mathrm{Cov}(d, z)\,\mathrm{Cov}(z, z)^{-1}$ eliminates this shift but requires a matrix inversion of $O(H^3)$, which undermines the benefits of locality described in this section. Intermediate designs, such as block diagonal approximations, are left for future work.

6.2 Noise Selection for Sparseness

The computational and memory requirements listed in Section 6.1 do not depend on the noise distribution. Furthermore, in this configuration, where the local slope is estimated by the kernel density estimation shown in Section 3.1, the analytical form of the noise distribution appears

Table 5. Effect of the noise distribution on final regression performance. Values report the mean ± standard deviation of the final MSE over three different seeds under Gaussian noise with $\sigma = 0.5$ and uniform noise centered at $c = 0$ with half-width $r = 1.0$. The Gaussian-noise results are reproduced from Table 2, and all other experimental conditions are unchanged.

Method	Gaussian ($\sigma = 0.5$, Table 2 reproduced)	Uniform ($r = 1.0$)
backprop	0.00057 ± 0.00010	0.00050 ± 0.00002
`cov_only`	0.09583 ± 0.00724	0.12688 ± 0.00534
`cov_deriv`	0.03464 ± 0.00558	0.05765 ± 0.00951
`cov_jac`	0.00056 ± 0.00006	0.00046 ± 0.00001
`cov_jac_full`	0.00057 ± 0.00009	0.00048 ± 0.00005

nowhere in inference or learning. That is, changing the noise distribution does not alter the network operations. We confirm this through a replication experiment using the identical protocol but replacing the noise with a uniform distribution $p(\xi) = \frac{1}{2r}\mathbf{1}\{|\xi - c| \le r\}$, where c is the center and r is the half width, with $c = 0$ and $r = 1.0$ in the experiment. As shown in Table 5, the final MSE of `cov_jac` and `cov_jac_full` matches backprop even with uniform noise, and the performance ranking among methods remains identical to Section 5.2. The agreement between the estimated slope and the analytical $\bar{\phi}'$ was also reproduced, yielding 0.054 compared to 0.058. Therefore, the noise source can be selected based on implementation cost rather than learning performance.

From this perspective, there are two essential reasons to choose uniform noise in digital hardware implementations. First, uniform random numbers can be generated using only an LFSR via shifts and XOR operations, eliminating the extra circuits required for Box Muller or CLT approximations needed for Gaussian noise. Second, the statistically equivalent activation function has compact support. With uniform noise, $F(d) = (d - c + r)/(2r)$ holds within the range $|d - c| < r$. From the general formula $\bar{\phi} = 2F(1 - F)$ in Section 3.1, the expected response degenerates into a strict parabola, and the local derivative simplifies to a linear expression.

$$\bar{\phi}(d) = \frac{1}{2}\left[1 - \left(\frac{d-c}{r}\right)^2\right]_+ \tag{26}$$

$$\bar{\phi}'(d) = -\frac{d-c}{r^2} \quad (|d - c| < r, \text{ otherwise } 0). \tag{27}$$

The expected response is strictly 0 when $|d - c| \ge r$, meaning that units far from the threshold maintain exactly zero spikes and learning statistics. With Gaussian noise, the firing probability is positive everywhere, meaning this sparsity cannot be achieved. Compact support directly leads to activity and power sparsity in event driven implementations. The simplicity of the analytical expressions is a secondary benefit because they are not used in this configuration employing the estimated slope.

Finally, noise is a design variable not only in its distribution but also in its spatial assignment. The noise field described in Section 3.2 implies that the noise assignment determines which units receive credit, providing design flexibility to suppress intra-layer correlations and directly couple update gating with power consumption gating. These designs and their evaluations are left for future work.

7 Discussion

The proposed learning rules assign credit using only forward statistics, without weight transport or a dedicated perturbation phase. Their applicability, however, faces several limitations. First, the evaluation is limited to small scale problems. The tasks consist of one dimensional regression and low dimensional benchmarks, while the network features two hidden layers and a scalar output, leaving multiclass classification and deep networks untested. Especially in deep networks, because the univariate mirror is a diagonal approximation ignoring intra-layer correlations, its estimation error may accumulate across layers through recursion, which remains an open question. Second, achieving backpropagation-level performance comes at a cost in computation and optimization. Each update requires T forward samplings, and reaching this performance demands the use of Adam. Although the state of Adam remains local to each weight to preserve locality, it increases the memory and computation required for hardware implementation. Third, the discussion on hardware suitability remains preliminary, leaving its implementation and empirical validation for future work.

8 Conclusion

This paper demonstrates that the structure of backpropagation can be reconstructed in a noise modulated neural network (NNN) solely from the covariance statistics of the forward fluctuations. The proposed `cov_jac` rule estimates weights from forward perturbations in crossing activations and recurses them layer wise. Furthermore, `cov_jac_full` estimates the readout error from the covariance, generalizing `cov_jac` to a configuration that completely eliminates analytical loss derivatives. Both rules approximate exact gradients without transporting or transposing weights, achieving accuracy comparable to backpropagation in both regression and classification tasks. These learning rules naturally integrate existing ideas that avoid weight transport into NNN dynamics without a dedicated perturbation phase, balancing biological plausibility and functionality. Additionally, because the required operations are confined to a few primitives centered around accumulators and comparators, the overall configuration demonstrates strong potential for hardware implementation. In conclusion, this paper shows that the NNN is a mathematical model that achieves a balance between biological plausibility and engineering applicability in neuromorphic computing.

Acknowledgement and Funding

This work was supported by the Japan Science and Technology Agency (JST) Fusion Oriented Research for Disruptive Science and Technology (FOREST) Program under Grant JPMJFR242H. This work was supported by the Japan Society for Promotion of Science (JSPS) KAKENHI Grant Number 25H02619.

Author contributions

Shuhei Ikemoto

Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Visualization, Writing – Original Draft, Writing – Review & Editing, Funding acquisition

Data availability

The data that support the findings of this study are openly available at the following URL: https://github.com/ikesyu/nnn-covariance-credit-assignment. The repository contains the complete source code of the Noise-modulated Neural Network library together with the scripts that reproduce all tables (Tables 2–5) and figures (Figures 2–5) reported in this paper, released under the MIT License. No external datasets were used in this study; all benchmark data are generated synthetically by the provided scripts with fixed random seeds.

References

[1] Luca Gammaitoni, Peter Hänggi, Peter Jung, and Fabio Marchesoni. Stochastic resonance. *Reviews of Modern Physics*, 70:223–287, Jan 1998.

[2] Frank Moss, Lawrence M Ward, and Walter G Sannita. Stochastic resonance and sensory information processing: a tutorial and review of application. *Clinical Neurophysiology*, 115(2):267–281, 2004.

[3] Ila R. Fiete and H. Sebastian Seung. Gradient learning in spiking neural networks by dynamic perturbation of conductances. *Physical Review Letters*, 97(4):048104, 2006.

[4] Ila R. Fiete, Michale S. Fee, and H. Sebastian Seung. Model of birdsong learning based on gradient estimation by dynamic perturbation of neural conductances. *Journal of Neurophysiology*, 98(4):2038–2057, 2007.

[5] Dejan Pecevski, Lars Buesing, and Wolfgang Maass. Probabilistic inference in general graphical models through sampling in stochastic networks of spiking neurons. *PLoS Computational Biology*, 7(12):e1002294, 2011.

[6] Gergő Orbán, József Fiser, Pietro Berkes, and Máté Lengyel. Neural variability and sampling-based probabilistic representations in the visual cortex. *Neuron*, 92(2):530–543, 2016.

[7] A. Aldo Faisal, Luc P. J. Selen, and Daniel M. Wolpert. Noise in the nervous system. *Nature Reviews Neuroscience*, 9(4):292–303, 2008.

[8] Mark D. McDonnell and Lawrence M. Ward. The benefits of noise in neural systems: bridging theory and experiment. *Nature Reviews Neuroscience*, 12(7):415–425, 2011.

[9] Shuhei Ikemoto, Fabio DallaLibera, and Koh Hosoda. Noise-modulated neural networks as an application of stochastic resonance. *Neurocomputing*, 277:29 – 37, 2018.

[10] Shuhei Ikemoto. Noise-modulated neural networks for selectively functionalizing sub-networks by exploiting stochastic resonance. *Neurocomputing*, 448:1–9, 2021.

[11] Shuhei Ikemoto and Fabio DallaLibera. Spatial partial functionalization of neural networks based on noise fields, 2026.

[12] Emre O. Neftci, Hesham Mostafa, and Friedemann Zenke. Surrogate gradient learning in spiking neural networks: Bringing the power of gradient-based optimization to spiking neural networks. *IEEE Signal Processing Magazine*, 36(6):51–63, 2019.

[13] Emre O. Neftci, Charles Augustine, Somnath Paul, and Georgios Detorakis. Event-driven random back-propagation: Enabling neuromorphic deep learning machines. *Frontiers in Neuroscience*, Volume 11 - 2017, 2017.

[14] Charlotte Frenkel, Martin Lefebvre, and David Bol. Learning without feedback: Fixed random learning signals allow for feedforward training of deep neural networks. *Frontiers in Neuroscience*, Volume 15 - 2021, 2021.

[15] Timothy P. Lillicrap, Adam Santoro, Luke Marris, Colin J. Akerman, and Geoffrey Hinton. Backpropagation and the brain. *Nature Reviews Neuroscience*, 21:335–346, 6 2020.

[16] Francis Crick. The recent excitement about neural networks. Technical report, 1989.

[17] Timothy P. Lillicrap, Daniel Cownden, Douglas B. Tweed, and Colin J. Akerman. Random synaptic feedback weights support error backpropagation for deep learning. *Nature Communications*, 7, 11 2016.

[18] Arild Nøkland. Direct feedback alignment provides learning in deep neural networks. 12 2016.

[19] J.F. Kolen and J.B. Pollack. Backpropagation without weight transport. In *Proceedings of 1994 IEEE International Conference on Neural Networks (ICNN'94)*, volume 3, pages 1375–1380 vol.3, 1994.

[20] Mohamed Akrout, Collin Wilson, Peter C. Humphreys, Timothy Lillicrap, and Douglas Tweed. Deep learning without weight transport. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

[21] J.C. Spall. Multivariate stochastic approximation using a simultaneous perturbation gradient approximation. *IEEE Transactions on Automatic Control*, 37(3):332–341, 1992.

[22] R. Benzi, A. Sutera, and A. Vulpiani. The mechanism of stochastic resonance. *Journal of Physics A: Mathematical and general*, 14:453–457, 1981.

[23] Roberto Benzi, Giorgio Parisi, Alfonso Sutera, and Angelo Vulpiani. Stochastic resonance in climatic change. *Tellus*, 34(1):10–16, 1982.

[24] K. Wiesenfeld and F. Moss. Stochastic resonance and the benefits of noise: from ice ages to crayfish and SQUIDs. *Nature*, 373(6509):33–36, 1995.

[25] Mark D McDonnell and Derek Abbott. What is stochastic resonance? definitions, misconceptions, debates, and its relevance to biology. *PLoS Comput Biol*, 5(5):e1000348, 2009.

[26] J.K. Douglass, L. Wilkens, E. Pantazelou, and F. Moss. Noise enhancement of information transfer in crayfish mechanoreceptors by stochastic resonance. *Nature*, 365(6444):337–340, 1993.

[27] J.E. Levin and J.P. Miller. Broadband neural encoding in the cricket cercal sensory system enhanced by stochastic resonance. *Nature*, 380:165–168, 1996.

[28] James J. Collins, Thomas T. Imhoff, and Peter Grigg. Noise-enhanced tactile sensation. *Nature*, 383(6603):770, 1996.

[29] Patrik Hoyer and Aapo Hyvärinen. Interpreting neural response variability as monte carlo sampling of the posterior. In S. Becker, S. Thrun, and K. Obermayer, editors, *Advances in Neural Information Processing Systems*, volume 15. MIT Press, 2002.

[30] Ralf M. Haefner, Pietro Berkes, and József Fiser. Perceptual decision-making as probabilistic inference by neural sampling. *Neuron*, 90(3):649–660, 2016.

[31] Pietro Berkes, Gergő Orbán, Máté Lengyel, and József Fiser. Spontaneous cortical activity reveals hallmarks of an optimal internal model of the environment. *Science*, 331(6013):83–87, 2011.

[32] Lars Buesing, Johannes Bill, Bernhard Nessler, and Wolfgang Maass. Neural dynamics as sampling: A model for stochastic computation in recurrent networks of spiking neurons. *PLOS Computational Biology*, 7(11):1–22, 11 2011.

[33] C. M. Bishop. Training with noise is equivalent to tikhonov regularization. *Neural Computation*, 7(1):108–116, 1995.

[34] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(56):1929–1958, 2014.

[35] Arvind Neelakantan, Luke Vilnis, Quoc V. Le, Ilya Sutskever, Lukasz Kaiser, Karol Kurach, and James Martens. Adding gradient noise improves learning for very deep networks, 2015.

[36] Wolfgang Maass. Networks of spiking neurons: The third generation of neural network models. *Neural Networks*, 10(9):1659–1671, 1997.

[37] Jun Haeng Lee, Tobi Delbruck, and Michael Pfeiffer. Training deep spiking neural networks using backpropagation. *Frontiers in Neuroscience*, Volume 10 - 2016, 2016.

[38] Friedemann Zenke and Tim P. Vogels. The remarkable robustness of surrogate gradient learning for instilling complex function in spiking neural networks, 3 2021.

[39] Yoshua Bengio, Nicholas Léonard, and Aaron Courville. Estimating or propagating gradients through stochastic neurons for conditional computation, 2013.

[40] Jean-Pascal Pfister, Taro Toyoizumi, David Barber, and Wulfram Gerstner. Optimal spike-timing-dependent plasticity for precise action potential firing in supervised learning. *Neural Computation*, 18:1318–1348, 06 2006.

[41] Danilo Jimenez Rezende and Wulfram Gerstner. Stochastic variational learning in recurrent spiking networks. *Frontiers in Computational Neuroscience*, Volume 8 - 2014, 2014.

[42] Armin Alaghi and John P. Hayes. Survey of stochastic computing. *Transactions on Embedded Computing Systems*, 12, 5 2013.

[43] Daniel Kunin, Aran Nayebi, Javier Sagastuy-Brena, Surya Ganguli, Jonathan Bloom, and Daniel Yamins. Two routes to scalable credit assignment without weight symmetry. In Hal Daumé III and Aarti Singh, editors, *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 5511–5521. PMLR, 13–18 Jul 2020.

[44] Yoshua Bengio. How auto-encoders could provide credit assignment in deep networks via target propagation, 2014.

[45] Dong-Hyun Lee, Saizheng Zhang, Asja Fischer, and Yoshua Bengio. Difference target propagation. In Annalisa Appice, Pedro Pereira Rodrigues, Vítor Santos Costa, Carlos Soares, João Gama, and Alípio Jorge, editors, *Machine Learning and Knowledge Discovery in Databases*, pages 498–515, Cham, 2015. Springer International Publishing.

[46] Maxence Ernoult, Fabrice Normandin, Abhinav Moudgil, Sean Spinney, Eugene Belilovsky, Irina Rish, Blake Richards, and Yoshua Bengio. Towards scaling difference target propagation by learning backprop targets. In *International Conference on Machine Learning*, pages 5968–5987. PMLR, 2022.

[47] Benjamin Scellier and Yoshua Bengio. Equilibrium propagation: Bridging the gap between energy-based models and backpropagation. *Frontiers in Computational Neuroscience*, Volume 11 - 2017, 2017.

[48] James C.R. Whittington and Rafal Bogacz. An approximation of the error backpropagation algorithm in a predictive coding network with local hebbian synaptic plasticity, 5 2017.

[49] Qianli Liao, Joel Z. Leibo, and Tomaso Poggio. How important is weight symmetry in backpropagation? In *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence*, pages 1837–1844. AAAI Press, 2016.

[50] Will Xiao, Honglin Chen, Qianli Liao, and Tomaso Poggio. Biologically-plausible learning algorithms can scale to large datasets. In *International Conference on Learning Representations (ICLR)*, 2019.

[51] Justin Werfel, Xiaohui Xie, and H. Sebastian Seung. Learning curves for stochastic gradient descent in linear feedforward networks. *Neural Computation*, 17(12):2699–2718, 2005.

[52] Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3-4):229–256, 1992.

[53] Atılım Güneş Baydin, Barak A. Pearlmutter, Don Syme, Frank Wood, and Philip H. S. Torr. Gradients without backpropagation. *arXiv preprint arXiv:2202.08587*, 2022.

[54] Mengye Ren, Jae Simon, and Raquel Urtasun. Scaling forward gradient with local losses. In *International Conference on Learning Representations (ICLR)*, 2023.

[55] Geoffrey Hinton. The forward-forward algorithm: Some preliminary investigations. *arXiv preprint arXiv:2212.13345*, 2022.

[56] Giorgia Dellaferrera and Gabriel Kreiman. Error-driven input modulation: Solving the credit assignment problem without a backward pass. In *International Conference on Machine Learning (ICML)*, pages 4937–4955. PMLR, 2022.

[57] Arild Nøkland and Lars Hiller Eidnes. Training neural networks with local error signals. In *International Conference on Machine Learning (ICML)*, pages 4750–4759. PMLR, 2019.

[58] Eugene Belilovsky, Michael Eghbal-zadeh, and Moustapha Cisse. Greedy layerwise learning of deep convolutional networks. In *International Conference on Machine Learning (ICML)*, pages 567–576. PMLR, 2019.

[59] Guillaume Bellec, Franz Scherr, Anand Subramanian, Darjan Salaj, Robert Legenstein, and Wolfgang Maass. A solution to the learning dilemma for recurrent networks of spiking neurons. *Nature Communications*, 11(1):3625, 2020.

[60] Naoki Hiratani, Yash Mehta, Timothy Lillicrap, and Peter E. Latham. On the stability and scalability of node perturbation learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 31929–31941, 2022.

[61] Sergey Bartunov, Adam Santoro, Blake Richards, Luke Marblestone, Geoffrey Hinton, and Timothy Lillicrap. Assessing the scalability of biologically-plausible cortical receptors. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 31, pages 9303–9313, 2018.

[62] Theodore H. Moskovitz, Ashok Litwin-Kumar, and L. F. Abbott. Feedback alignment in deep convolutional networks. *arXiv preprint arXiv:1812.06488*, 2018.